

Software Design Document (SDD)

Module UART Driver

Lie Detector — GSR & Heart Rate Sensor System

Version	2.0.0
Team	Kheara Kieley · Mauricio Vela · Ximena Cedillo · Michelle Urbina
Date	03/06/2026
Institution	CETYS Universidad

1. Introduction

1.1 Purpose

This Software Design Document (SDD) describes the updated design and architecture of the UART driver module for the STM32F411RE microcontroller, adapted for the Lie Detector project (Lab Assignment Final). The document covers register-level configuration of USART2, baud rate calculation, the polling transmit loop, and the Serial module that formats and sends multi-field telemetry packets to the Python GUI application (`lie_detector_gui.py`) running on a host PC.

1.2 Scope

The UART driver configures USART2 on the STM32F411RE for asynchronous, TX-only serial communication at 115200 baud. It relies on the 16 MHz HSI clock (default after reset) and PA2 as the transmit pin in Alternate Function 7. In this project the Serial module wraps the driver and formats structured telemetry frames containing GSR and heart rate data. These frames are transmitted to `lie_detector_gui.py` via the ST-Link Virtual COM Port every ~500 ms. The frame format is pipe-delimited plain text.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
AF7	Alternate Function 7 — routes PA2 to USART2_TX.
APB1	Advanced Peripheral Bus 1 — clocks USART2 on STM32F4.
BRR	Baud Rate Register — holds the integer and fractional divider.
CMSIS	Cortex Microcontroller Software Interface Standard.
CR1	USART Control Register 1 — contains TE and UE bits.
DR	USART Data Register — the transmit holding register.
FR	Functional Requirement — traceability label from the SRS.
GSR	Galvanic Skin Response — skin conductance sensor read via ADC.
HR	Heart Rate — optical pulse sensor value read via ADC.
HSI	High-Speed Internal oscillator — 16 MHz default clock.
PA2	GPIO pin A2 — USART2_TX via Alternate Function 7.
SR	USART Status Register — contains TXE and TC flags.
TE	Transmitter Enable bit (CR1 bit 3).
TXE	Transmit Data Register Empty flag (SR bit 7).
UART	Universal Asynchronous Receiver-Transmitter.
USART2	USART2 peripheral, mapped to PA2 (TX), routed to ST-Link VCP.
VCP	Virtual COM Port — USB serial interface from ST-Link.

BPM	Beats Per Minute — computed heart rate value.
Rpiel	Skin resistance value derived from GSR ADC reading (Ohms).
LieP	Lie probability index computed by the embedded algorithm (-1 = uncalibrated).
Estado	System state string (e.g., SIN_CAL, CALIBRANDO, MIDIENDO).

1.4 Requirements

The functional requirements this module must satisfy are listed below. All are defined in SRS-UART_Driver for the Lie Detector project.

Req. ID	Description
FR-1	The driver shall enable the USART2 peripheral clock via RCC_APB1ENR. Implemented in <code>uart_init()</code> .
FR-2	The driver shall set the baud rate to 115200 using $BRR = fCK / \text{baud_rate}$ with a 16 MHz clock. Implemented in <code>uart_init()</code> .
FR-3	The driver shall enable the USART transmitter (TE) and peripheral (UE) by writing CR1. Implemented in <code>uart_init()</code> .
FR-4	The driver shall wait until TXE flag is set before writing to DR. Implemented in <code>uart_write()</code> .
FR-5	The driver shall write the byte to USART2->DR to initiate transmission. Implemented in <code>uart_write()</code> .
FR-6	The Serial module shall format telemetry frames with the fields Raw, Filt, Rpiel, HR, BPM, LieP, and Estado separated by ' '. Implemented in <code>serial_printf()</code> .
FR-7	The Serial module shall transmit one complete telemetry frame per sampling period (~500 ms). Implemented by <code>main.c</code> calling <code>serial_printf()</code> .

2. System Architecture

2.1 High-Level Design

The Lie Detector project adds a structured telemetry path on top of the GSR and heart rate sensor infrastructure. The software stack for the UART subsystem has four levels:

Layer	Description
Application Layer	main.c initializes all modules and calls serial_printf() inside the timer loop (~500 ms intervals) to send telemetry frames containing GSR and HR measurements to lie_detector_gui.py.
Serial Layer (serial.c)	Wraps uart_init() and uart_write(). Provides serial_printf() which formats the pipe-delimited telemetry frame: Raw Filt Rpiel HR BPM LieP Estado. Uses a 128-byte internal buffer.
UART Driver Layer (uart.c)	Owns USART2 register configuration (PA2 alternate function, BRR, CR1) and provides a two-function API: uart_init and uart_write.
Hardware Layer	STM32F411RE USART2 peripheral registers and PA2 GPIO, accessed through CMSIS stm32f4xx.h structures.

2.2 Module Interfaces

The UART driver is declared in uart.h and depends on gpio_driver.h for gpio_initPort() and gpio_setAlternateFunction(). The driver calls gpio_initPort(GPIOA) to enable the GPIOA clock and gpio_setAlternateFunction(GPIOA, 2, 7) to route PA2 to AF7 (USART2_TX).

The Serial module is declared in serial.h. It includes uart.h and utils.h. serial_printf() uses a static 128-byte buffer and implements a variadic format loop with va_list directly.

Key CMSIS register references: USART2->CR1 (TE, UE bits), USART2->BRR (baud rate divider = 139), USART2->SR (TXE flag), USART2->DR (transmit data register), RCC->APB1ENR (USART2EN bit), GPIOA->MODER and GPIOA->AFR[0] (PA2 alternate function via gpio driver).

2.3 Telemetry Frame Format

Each frame transmitted to the Python GUI follows the pipe-delimited structure below, terminated with a newline character (\n):

Field	Example Value	Description
Raw	821	Raw ADC value from GSR sensor (12-bit, 0–4095).
Filt	821	Low-pass filtered GSR ADC value.
Rpiel	41103	Computed skin resistance in Ohms derived from filtered GSR.
HR	2280	Raw heart rate ADC sample value.

BPM	82	Computed heart rate in beats per minute.
LieP	-1	Lie probability index. -1 = uncalibrated; 0–100 = calibrated percentage.
Estado	SIN_CAL	System state string: SIN_CAL, CALIBRANDO, or MIDIENDO.

Example complete frame:

Raw:821|Filt:821|Rpiel:41103|HR:2280|BPM:82|LieP:-1|Estado:SIN_CAL\n

3. Detailed Design

3.1 Class Diagram

The class diagram for the UART driver in the Lie Detector project maintains the same structure as v1.0.0. The `uart_driver` class provides `uart_init()` and `uart_write()`. The `serial` class adds `serial_printf()` and depends on `utils` for integer-to-ASCII conversion. The `sensor` and `adc_driver` classes are shown to illustrate that sensor data flows upward to `serial_printf()` via `main.c`.

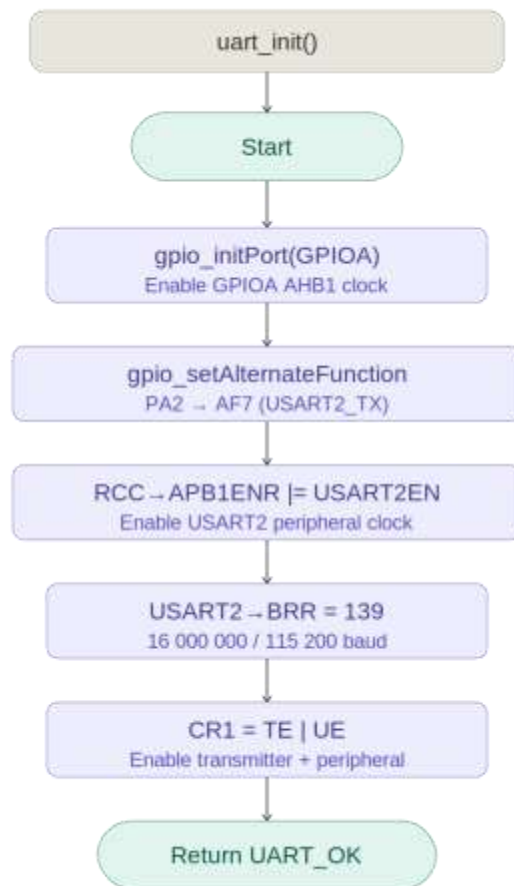
Modules present in the project relevant to UART: `main` → `serial` → `uart_driver` → `CMSIS_USART2`. The `sensor`, `adc_driver`, and `tim_driver` modules feed data to `main.c` which passes it to `serial_printf()` for transmission.

3.2 Module Behavior

The eight figures below cover the UART initialization sequence, the transmit polling loop, the `serial_printf` formatting path, and the driver state machine. The behavior is unchanged from v1.0.0 at the driver level. The Serial layer has been updated to produce the pipe-delimited multi-field frame format described in Section 2.3.

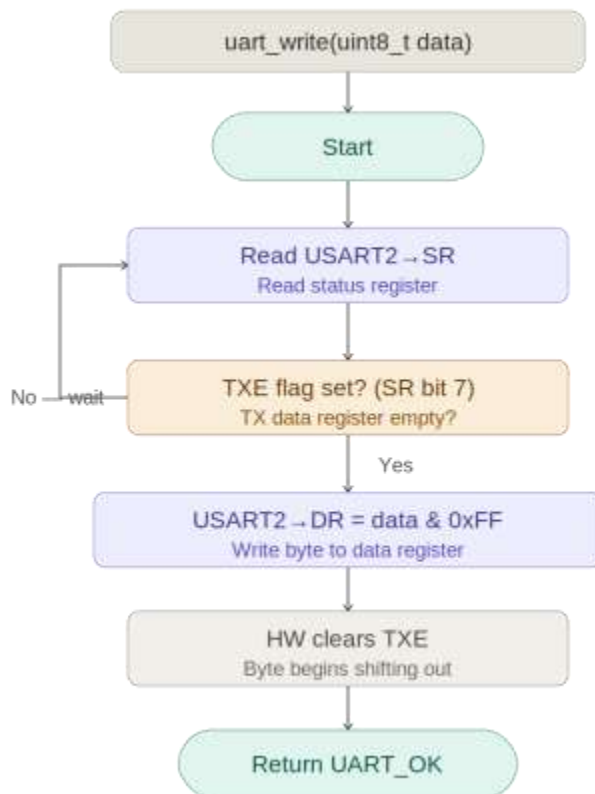
3.2.1 `uart_init` Sequence

Caller invokes `uart_init()`. Driver calls `gpio_initPort(GPIOA)`, then `gpio_setAlternateFunction(GPIOA, PA2, AF7)`. Hardware writes: `APB1ENR |= USART2EN`, `BRR = 139`, `CR1 = TE | UE`. Returns `UART_OK`.



3.2.2 uart_write Sequence

Caller invokes `uart_write(data)`. Driver polls `USART2->SR` until `TXE` (bit 7) is set, then writes data & `0xFF` to `USART2->DR`. Hardware clears `TXE` automatically. Returns `UART_OK`. No timeout mechanism — blocks indefinitely if USART is not enabled.

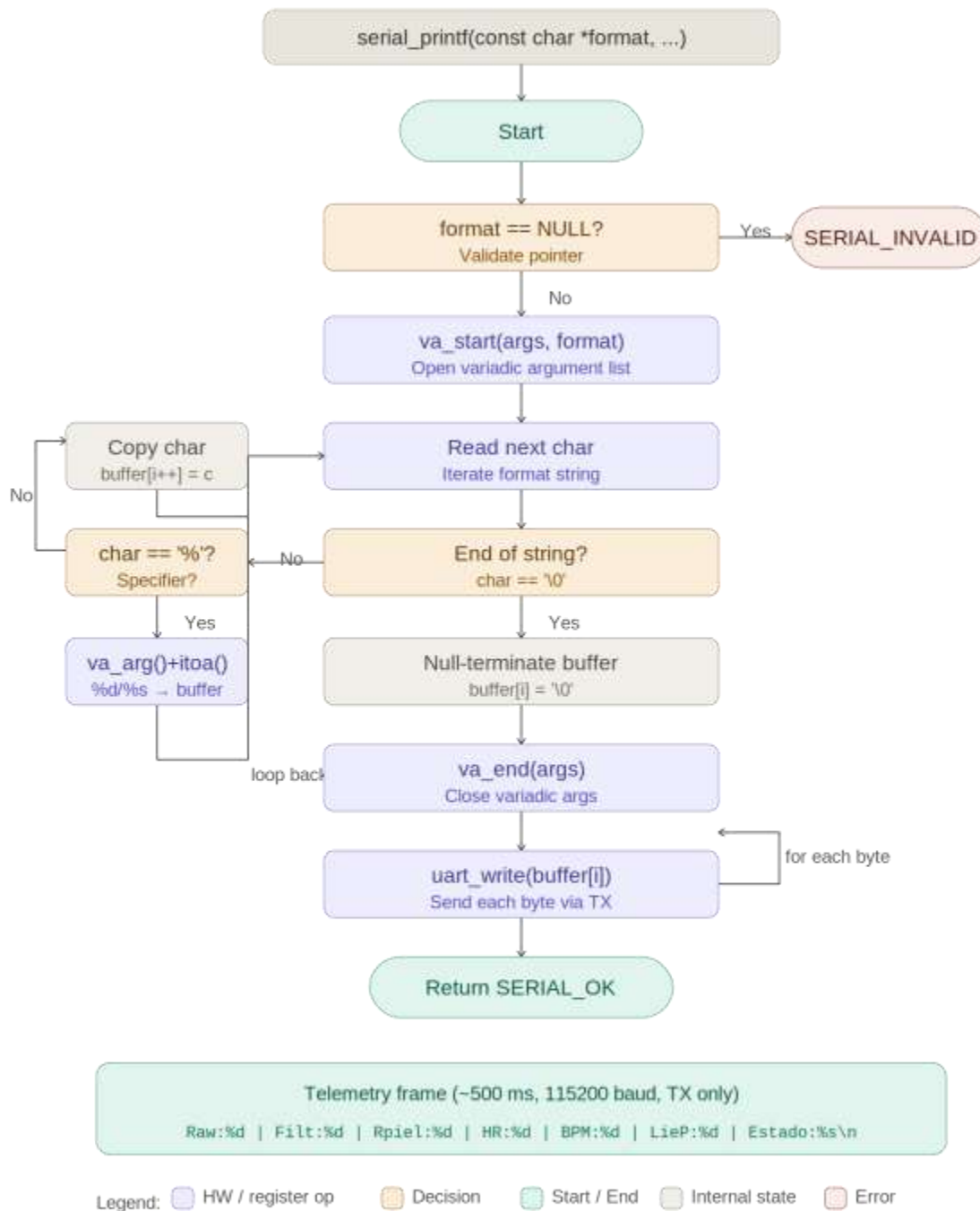


3.2.3 serial_printf Sequence (Updated for Lie Detector)

`main.c` calls `serial_printf()` once per ~500 ms timer tick with the format string:

```
"Raw:%d|Filt:%d|Rpiel:%d|HR:%d|BPM:%d|LieP:%d|Estado:%s\n"
```

`serial_printf()` opens a `va_list`, iterates through the format string, dispatches `%d` specifiers to `utils_itoa()` and `%s` specifiers by copying characters. The result accumulates in a 128-byte static buffer. Each byte is then sent by `uart_write()`. Output is silently truncated at 127 characters.



3.3 UART Driver Description

The UART driver is a two-function module: `uart_init()` configures the hardware once at startup, and `uart_write()` sends one byte at a time. There are no private helper functions. The implementation is intentionally minimal.

During initialization the driver calls `gpio_initPort()` to enable the GPIOA AHB1 clock, then `gpio_setAlternateFunction()` to set PA2 MODER to 0b10 and write AF7 into GPIOA->AFR[0] bits [11:8]. The USART2 APB1 clock is enabled by setting RCC->APB1ENR bit 17. BRR is written as $16000000 / 115200 = 139$ (0x008B). Finally CR1 is set to USART_CR1_TE | USART_CR1_UE.

`uart_write()` is a blocking call. It polls SR until TXE (bit 7) is set, confirming the hardware shift register is ready, then writes the byte to DR. Using TXE rather than TC allows back-to-back byte writes for better throughput when sending full telemetry frames.

In the Lie Detector project the typical transmission sequence per 500 ms tick is: `main.c` reads GSR and HR data from the sensor module, computes Rpiel, BPM, and LieP, then calls `serial_printf()` with all seven fields. The formatted 50–60 byte frame is transmitted byte by byte through `uart_write()` at 115200 baud, appearing in `lie_detector_gui.py` as one complete line per refresh cycle.

4. Application Programming Interfaces

4.1 Data Types and Macros

`uart_status_t`: return type for `uart_init()` and `uart_write()`. Three values: `UART_OK` (0) — operation succeeded; `UART_ERROR` (1) — generic failure; `UART_INVALID` (2) — invalid argument.

`serial_status_t`: return type for `serial_init()` and `serial_printf()`. Values: `SERIAL_OK` (0), `SERIAL_ERROR` (1), `SERIAL_INVALID` (2).

Macros defined in `uart.h`: `UART_SYSTEM_CLOCK_HZ = 16000000UL`; `UART_BAUD_RATE = 115200UL`. Private constants in `uart.c`: `UART_TX_PORT = GPIOA`, `UART_TX_PIN = 2U`, `UART_TX_AF = 7U`.

Macro defined in `serial.h`: `SERIAL_BUFFER_SIZE = 128U` — maximum formatted string length per `serial_printf()` call. With seven telemetry fields the typical frame length is 50–65 bytes, well within the limit.

4.2 Error Codes

Code	Definition
<code>UART_OK</code> (0)	The operation completed successfully.
<code>UART_ERROR</code> (1)	A generic failure. Reserved for future use.
<code>UART_INVALID</code> (2)	An argument was out of range. Reserved for future parameter validation.
<code>SERIAL_OK</code> (0)	Formatting and transmission completed without errors.
<code>SERIAL_ERROR</code> (1)	An internal error occurred in the serial layer.
<code>SERIAL_INVALID</code> (2)	The format string passed to <code>serial_printf()</code> was <code>NULL</code> .

4.3 Function Definitions

`uart_init`

Function Name	<code>uart_init</code>
Syntax	<code>uart_status_t uart_init(void)</code>
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	<code>uart_status_t</code> — always returns <code>UART_OK</code> in the current implementation.

Description	Configures PA2 as USART2_TX (AF7), enables the USART2 APB1 clock, sets BRR to 139 (16 MHz / 115200), and writes CR1 = TE UE.
Constraints	Assumes a 16 MHz HSI clock. Does not configure RX.
Available via	uart.h

uart_write

Function Name	uart_write
Syntax	uart_status_t uart_write(uint8_t data)
Parameters (in)	data – uint8_t: the byte to transmit.
Parameters (inout)	None
Parameters (out)	None
Return value	uart_status_t – always returns UART_OK.
Description	Busy-waits until TXE (SR bit 7) is set, then writes data & 0xFF to USART2->DR.
Constraints	Blocks indefinitely if the USART peripheral is not enabled or if the transmitter stalls. No timeout mechanism is implemented.
Available via	uart.h

serial_init

Function Name	serial_init
Syntax	serial_status_t serial_init(void)
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	serial_status_t – SERIAL_OK on success.
Description	Calls uart_init() to configure USART2 at 115200 baud.
Constraints	None.
Available via	serial.h

serial_printf

Function Name	serial_printf
Syntax	serial_status_t serial_printf(const char *format, ...)

Parameters (in)	format – printf-style format string. Supported: %d, %u, %x, %s, %c, %%.
Parameters (inout)	None
Parameters (out)	None
Return value	serial_status_t – SERIAL_OK on success, SERIAL_INVALID if format is NULL.
Description	Formats the string into an internal 128-byte buffer using va_list and utils_itoa(). In the Lie Detector project the primary call is: serial_printf("Raw:%d Filt:%d Rpiel:%d HR:%d BPM:%d LieP:%d Estado:%s\n", raw, filt, rpiel, hr, bpm, lieP, estado) Sends each byte with uart_write(). Null-terminates before transmitting.
Constraints	Output truncated at 127 characters. Does not support field width, precision, or padding specifiers.
Available via	serial.h

4.4 Internal Function Definitions

The UART driver has no private helper functions. The Serial module formatting logic is inline within serial_printf(). The utils module provides static helpers (utils_printString, utils_printInt) documented in the Utils module SDD.

5. Future Enhancements

- Receive support: add `uart_read()` and configure PA3 as USART2_RX (AF7) to allow `lie_detector_gui.py` to send calibration commands or reset signals to the embedded system.
- Interrupt-driven transmit: replace the polling loop in `uart_write()` with a circular TX buffer and the USART2 TXE interrupt, freeing the CPU between byte transmissions.
- DMA transmit: connect USART2 TX to DMA1 Stream 6 Channel 4 to send entire telemetry frames without CPU involvement.
- Configurable baud rate: add `uart_setBaudRate(uint32_t baud)` to recalculate BRR at runtime.
- Frame validation: add a checksum or CRC field to the telemetry frame so `lie_detector_gui.py` can detect corrupted packets.
- Binary protocol: replace the pipe-delimited ASCII frame with a compact binary packet to reduce per-frame byte count and parsing overhead on the Python side.

6. References

- [1] Gbati, O. (2023). Embedded Systems Programming with the STM32 Microcontroller. Independently published.
- [2] Mazidi, M., Chen, S., & Naimi, S. (2020). The STM32F0 ARM Microcontroller and Embedded Systems. Microdigitaled.
- [3] Deitel, P., & Deitel, H. (2016). C How to Program (8th ed.). Pearson.
- [4] STMicroelectronics. (2016). RM0383: STM32F411 reference manual (Rev 3). https://www.st.com/resource/en/reference_manual/rm0383-stm32f411xce-advanced-armbased-32bit-mcus-stmicroelectronics.pdf
- [5] IEEE Std 1016-2009, IEEE Standard for Information Technology — Systems Design — Software Design Descriptions.